AIML4002 INTRODUCTION TO LLM

23DIT024

PRACTICAL 2

```
!pip install tokenizers sentencepiece nltk -q
```

```
import os
import math
from collections import Counter

from tokenizers import Tokenizer
from tokenizers.models import BPE, WordPiece
from tokenizers.trainers import BpeTrainer, WordPieceTrainer
from tokenizers.pre_tokenizers import Whitespace
```

```
corpus = [
    "Natural language processing is a fascinating field.",
    "Machine learning helps computers understand language.",
    "Language models predict the next word.",
    "Deep learning improves language understanding.",
    "Artificial intelligence powers modern applications.",
    "Natural language processing includes tokenization.",
    "WordPiece and BPE are subword tokenization methods.",
    "Vocabulary plays an important role in NLP."
]
```

```
with open("corpus.txt", "w") as f:
    for sentence in corpus:
        f.write(sentence + "\n")
```

```
bpe_tokenizer = Tokenizer(BPE())
bpe_tokenizer.pre_tokenizer = Whitespace()

trainer = BpeTrainer(
    vocab_size=80,
    special_tokens=["[PAD]", "[UNK]"]
)

bpe_tokenizer.train(["corpus.txt"], trainer)
```

```
wp_tokenizer = Tokenizer(WordPiece(unk_token="[UNK]"))
wp_tokenizer.pre_tokenizer = Whitespace()

trainer = WordPieceTrainer(
    vocab_size=80,
    special_tokens=["[PAD]", "[UNK]"]
)

wp_tokenizer.train(["corpus.txt"], trainer)
```

```
test_sentence = "Natural language models improve intelligence"
```

```
bpe_output = bpe_tokenizer.encode(test_sentence)

print("BPE Tokens")
print(bpe_output.tokens)
```

```
BPE Tokens
['Nat', 'ural', 'language', 'mo', 'de', 'l', 's', 'im', 'pro', 'v', 'e', 'in', 't', 'el', 'l', 'i', 'ge', 'n', 'ce']
```

```
wp_output = wp_tokenizer.encode(test_sentence)

print("WordPiece Tokens")
print(wp_output.tokens)
```

```
WordPiece Tokens
['N', '##at', '##u', '##r', '##a', '##l', 'language', 'm', '##o', '##de', '##l', '##s', 'i', '##m', '##p', '##r', '##o', '##
```

```
print("BPE Vocabulary Size :", len(bpe_tokenizer.get_vocab()))
print("WordPiece Vocabulary Size :", len(wp_tokenizer.get_vocab()))
```

```
BPE Vocabulary Size : 80
WordPiece Vocabulary Size : 80
```

```
words = []

for sentence in corpus:
    words.extend(sentence.lower().split())

unique_words = set(words)

print("Unique Words :", len(unique_words))
```

```
Unique Words : 42
```

```
covered = 0

for word in unique_words:
    tokens = bpe_tokenizer.encode(word).tokens
    if "[UNK]" not in tokens:
        covered += 1

coverage = covered / len(unique_words)

print("BPE Vocabulary Coverage")
print(round(coverage*100,2),"%")
```

```
BPE Vocabulary Coverage
100.0 %
```

```
covered = 0

for word in unique_words:
    tokens = wp_tokenizer.encode(word).tokens
    if "[UNK]" not in tokens:
        covered += 1

coverage = covered / len(unique_words)

print("WordPiece Vocabulary Coverage")
print(round(coverage*100,2),"%")
```

```
WordPiece Vocabulary Coverage
100.0 %
```

```
bpe_tokens = []

for sentence in corpus:
    bpe_tokens.extend(
        bpe_tokenizer.encode(sentence).tokens
    )

bpe_counts = Counter(bpe_tokens)
```

```
wp_tokens = []

for sentence in corpus:
    wp_tokens.extend(
        wp_tokenizer.encode(sentence).tokens
    )

wp_counts = Counter(wp_tokens)
```

```
N = sum(bpe_counts.values())
V = len(bpe_counts)

def bpe_probability(word):

    count = bpe_counts[word]

    return (count+1)/(N+V)
```

```
N2 = sum(wp_counts.values())
V2 = len(wp_counts)

def wp_probability(word):
```

```
    count = wp_counts[word]

    return (count+1)/(N2+V2)
```

```
def perplexity(tokens, probability_function):

    log_prob = 0

    for token in tokens:

        p = probability_function(token)

        log_prob += math.log2(p)

    return 2 ** (-log_prob/len(tokens))
```

```
tokens = bpe_tokenizer.encode(test_sentence).tokens

pp = perplexity(tokens,bpe_probability)

print("BPE Perplexity")
print(pp)
```

```
BPE Perplexity
57.67291207508894
```

```
tokens = wp_tokenizer.encode(test_sentence).tokens

pp = perplexity(tokens,wp_probability)

print("WordPiece Perplexity")
print(pp)
```

```
WordPiece Perplexity
39.53275040335039
```

```
print("\nComparison")

print("-------------------------------")

print("BPE Vocabulary :",len(bpe_tokenizer.get_vocab()))

print("WordPiece Vocabulary :",len(wp_tokenizer.get_vocab()))

print("-------------------------------")

print("BPE Perplexity :",perplexity(
    bpe_tokenizer.encode(test_sentence).tokens,
    bpe_probability))

print("WordPiece Perplexity :",perplexity(
    wp_tokenizer.encode(test_sentence).tokens,
    wp_probability))
```

```

Comparison
-------------------------------
BPE Vocabulary : 80
WordPiece Vocabulary : 80
-------------------------------
BPE Perplexity : 57.67291207508894
WordPiece Perplexity : 39.53275040335039
```